DevOps

Cloud Native
Introduction

Bibliography

Laszewski, Tom; Arora, Kamal; Farr, Erik; Zonooz, Piyum. Cloud Native Architectures: Design high-availability and cost-effective applications for the cloud (p. 8). Packt Publishing

AWS Definition

"Cloud computing is the on-demand delivery of compute power, database storage, applications, and other IT resources through a cloud services platform via the internet with pay-as-you-go pricing."

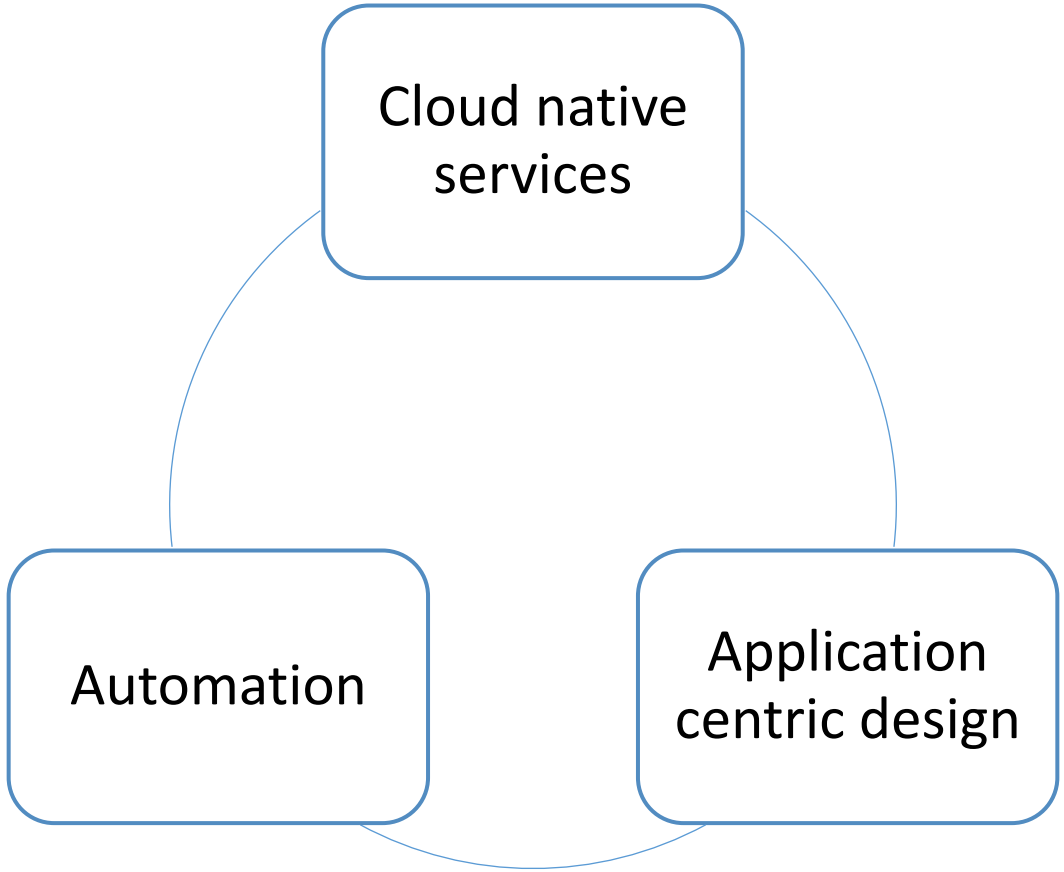
Embrace cloud computing services to design solutions

Extreme automation at scale

What does it mean?

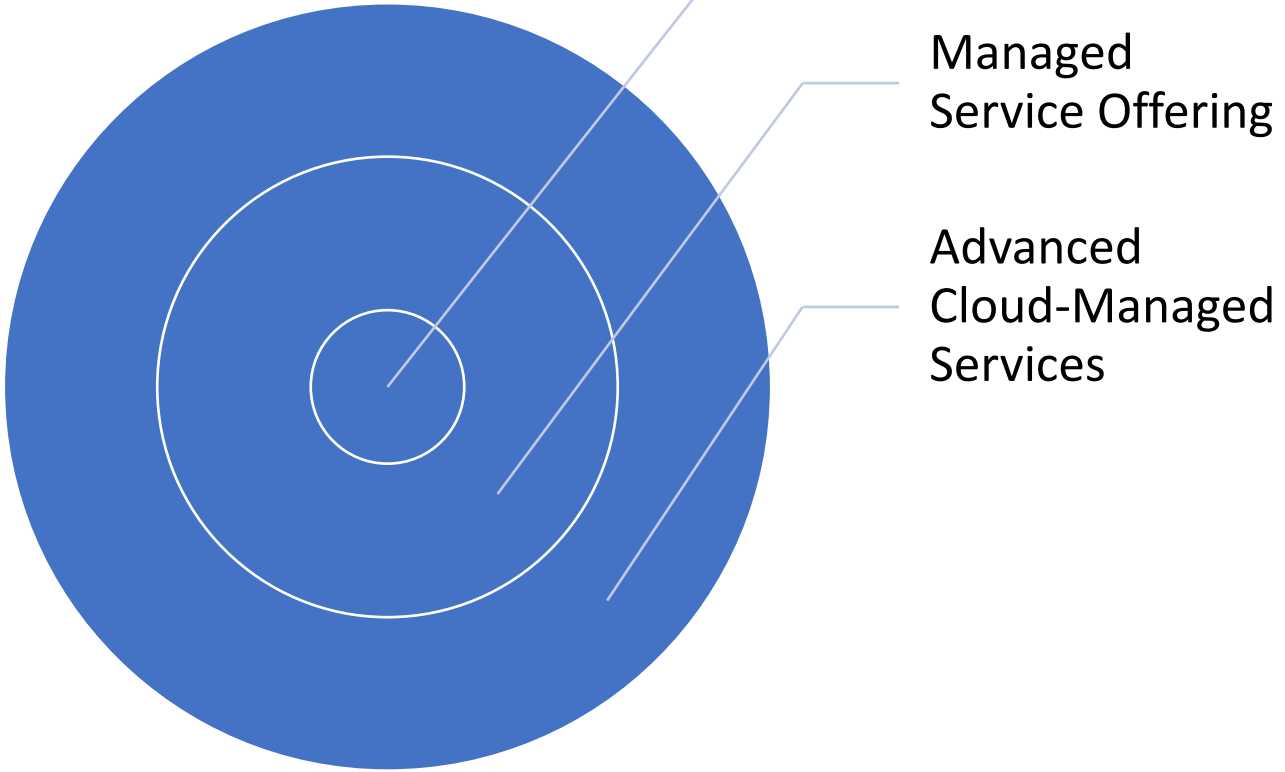
Cloud Native Maturity Model

The Three Axes



Axis 1 – Cloud Native Services

Incorporation of cloud services from basic building blocks to the most advanced, cutting-edge technologies



Have a look at AWS services

https://aws.amazon.com/products

Cloud Native Service Building Blocks

Infrastructure building blocks: compute, storage, networking and monitoring

Start with base line infrastructure: virtual server instances, block disk storage, object storage, fiber lines and VPNs, load balancers, cloud API monitoring, and instance monitoring

Bare minimum required to develop a cloud native architecture

Enables lift-and-shift migration model

Gain experience with how cloud works: security, deployment, networking, etc.

Key learning

How does cloud impact design: horizontal scaling vs vertical scaling

How the cloud vendor operates and group their services in specific locations and the interaction between the groupings to design high availability and high disaster recovery through architecture

The cloud approach to storage and the ability to offload processing to cloud services that scale efficiently and natively on the platform is a critical approach to designing architectures

Adoption of cloud services building blocks is critical for companies that are beginning their cloud journey

Cloud vendor managed service offering

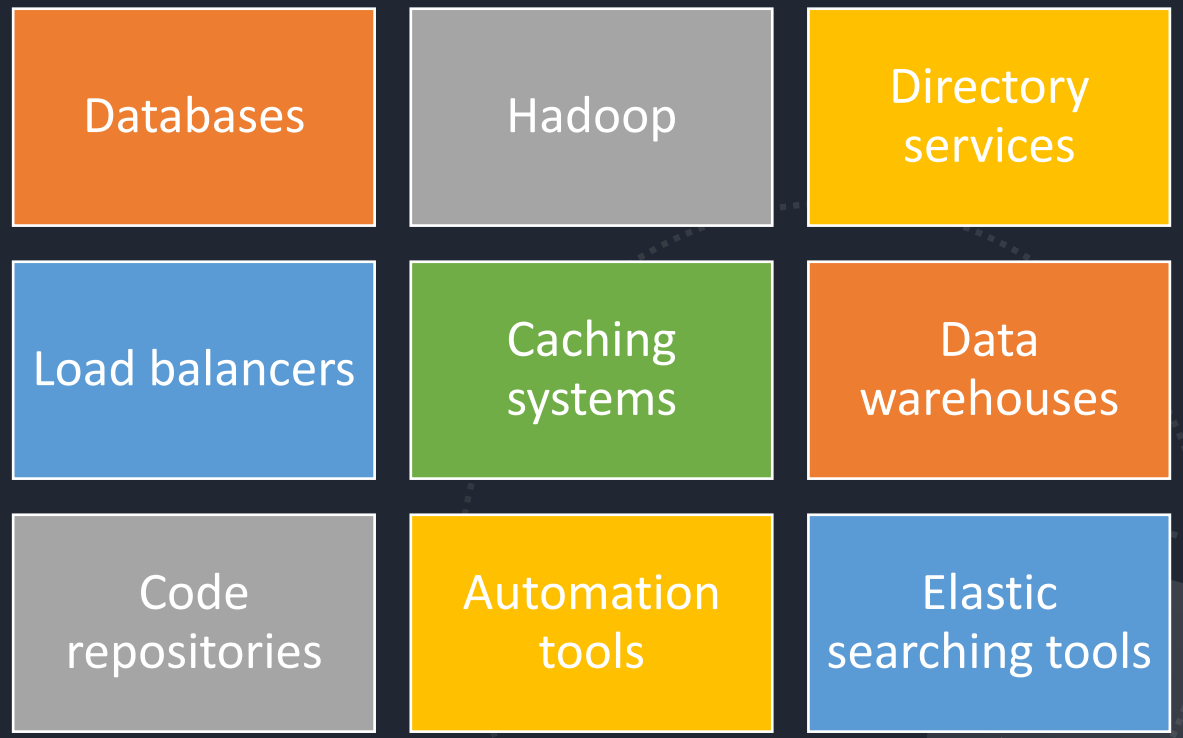
Undifferentiated heavy lifting describes a large majority of the IT operations for enterprise companies, and is a critical selling point to using the cloud

Cloud vendors do have a core competency in technology innovation and operations at a scale that most companies could never dream of

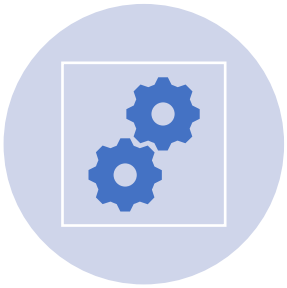
Cloud vendors have matured their services to include managed offerings, where they own the management of all aspects of the service and the consumer only needs to develop the business logic or data being deployed to the service

This will allow the undifferentiated heavy lifting to be shifted from the company to the cloud vendor, and allow that company to dedicate significantly more resources to creating business value (their differentiators)

Examples of Managed Services Offer



Benefits of Managed Services



Agility - allow the teams to implement the architecture quickly, and begin the process of testing the applications that will run in that environment

Ability to think bigger without being constrained by the undifferentiated heavy lifting

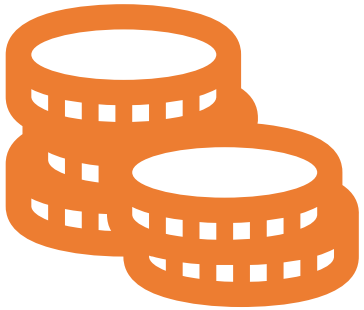
Advanced Cloud Native Managed Services

- More and more services that are managed by the vendor at scale and security, and reduces the cost, complexity, and risk to the customer
- **Re-engineering existing technology platforms** to not only deal with a specific technical problem, but by also doing it with the cloud value proposition in mind (*e.g., Amazon Aurora - a global-scale relational database service built for the cloud - MySQL, PostgreSQL*)
- **Serverless computing** – cloud vendors are removing undifferentiated heavy lifting away from not only operations, but also the development cycle

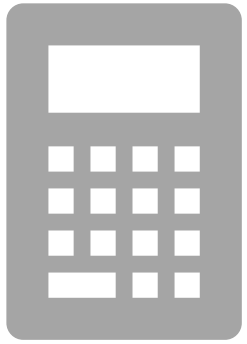
Serverless computing as seen be AWS

"Serverless computing allows you to build and run applications and services without thinking about servers. Serverless applications don't require you to provision, scale, and manage any servers. You can build them for virtually any type of application or backend service, and everything required to run and scale your application with high availability is handled for you."

Serverless Model



Consumption based-pricing

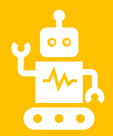


Common metrics: the length of time a function runs or, the amount of transactions

Benefits of Advanced Cloud Native Managed Services

Enable companies to develop some of the most advanced cloud native architectures

Focus on the business logic and not on how to achieve required scale



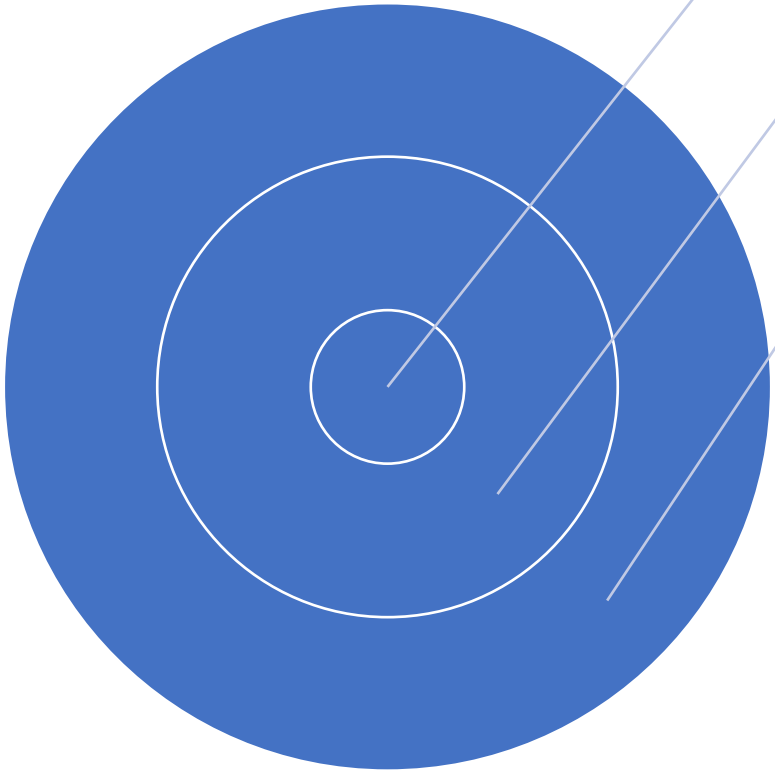
Currently, the cutting edge of cloud computing is the offerings being released in the artificial intelligence, machine learning, and deep learning space

Axes 2 – Application Centric Design

How the application itself will be designed and architected

Architecture patterns that lead to cloud native architectures

Mature, sophisticated, and robust cloud native architecture that is ready to continue evolving as the world of cloud computing expands



12-Factor App

Microservices

Cloud Native Design

Twelve-factor App Design Principles

A methodology for building software-as-a-service applications

Base building blocks for designing scalable and robust cloud native applications

Designing applications that minimize the time and cost of adding new developers, cleanly interoperate with the environment, can be deployed to cloud vendors, minimize divergence between environment, and allow for scaling of the application

https://12factor.net/

Factor #	Factors	Description
1	Code Base	One code base tracked in revision control, many deploys.
2	Dependencies	Explicitly declare and isolate dependencies
3	Config	Store config in the environment
4	Backing service	Treat backing service as attached resources
5	Build, release, run	Strictly separate build and run stages
6	Process	Execute the app as one or more stateless processes
7	Port binding	Export services through port binding, no need for runtime injection of a webserver
8	Concurrency	Scale-out through the process model
9	Disposability	Maximize robustness with fast startup and graceful shutdown
10	Dev/prod parity	Keep development, staging, and production as similar as possible
11	Logs	Treat logs as event streams
12	Admin processes	Run admin/management tasks as on-off processes

Additional factors to consider

API first - allows for consumer and service developers to work in parallel

Telemetry - monitoring our microservices, containers, and env help us to scale, self-heal and manage alerts for end-users and platform operators

Authentication - make sure all security policies are in place

https://domenicoluciani.com/2021/10/30/15-factor-app.html

Monolithic, SOA, and Microservices Architecture

Architecture design patterns are always evolving to take advantage of the newest technology innovations

Monolithic

Monolithic architectures were popular, often due to the cost of physical resources and the slow velocity in which applications were developed and deployed

Eventually, these monolithic applications needed to communicate with each other to share data or execute functions that the other systems contained => SOA

SOA

SOA - allowed design teams to create smaller application components, implement middleware components to mediate the communication, and isolate the ability to access the components, except through specific endpoints

Service-oriented architectures consist of two or more components which provide their services to other services through specific communication protocols

As the complexity of these different protocols and services grew, using an enterprise service bus (ESB) became increasingly common as a mediation layer between services

ESB => new complexity (deployment coordination)

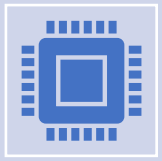
Monolithic vs SOA

SOA reduces blast radius

SOA requires mores complex deployments => slowed the pace of the business requirements

Microservices

Appeared as cloud computing became more common and the constraints of the on-premises environments began to fade a way



The microservices architecture style takes the distributed nature of SOA and breaks those services up into even more discrete and loosely coupled application functions

Microservices Characteristics

Reduced blast radius

Dramatically increased velocity of applications

Requires good DevOps practice

The most mature cloud native architecture

Increased innovation and design patterns that take advantage of the cloud

Focus on business logic

Cloud Native Design Considerations

Instrumentation

Ability to monitor and measure the performance of the application in real time

Ability of the application to be self-aware of latency conditions, component failures due to system faults, and other characteristics that are important to a specific business application

Critical to other design considerations

Enable long-term benefits

Security

Take advantage of cloud vendor security services and third-party security services

Security in layers

Goal: harden the posture of the application and reduce the blast radius in the event of an attack or breach

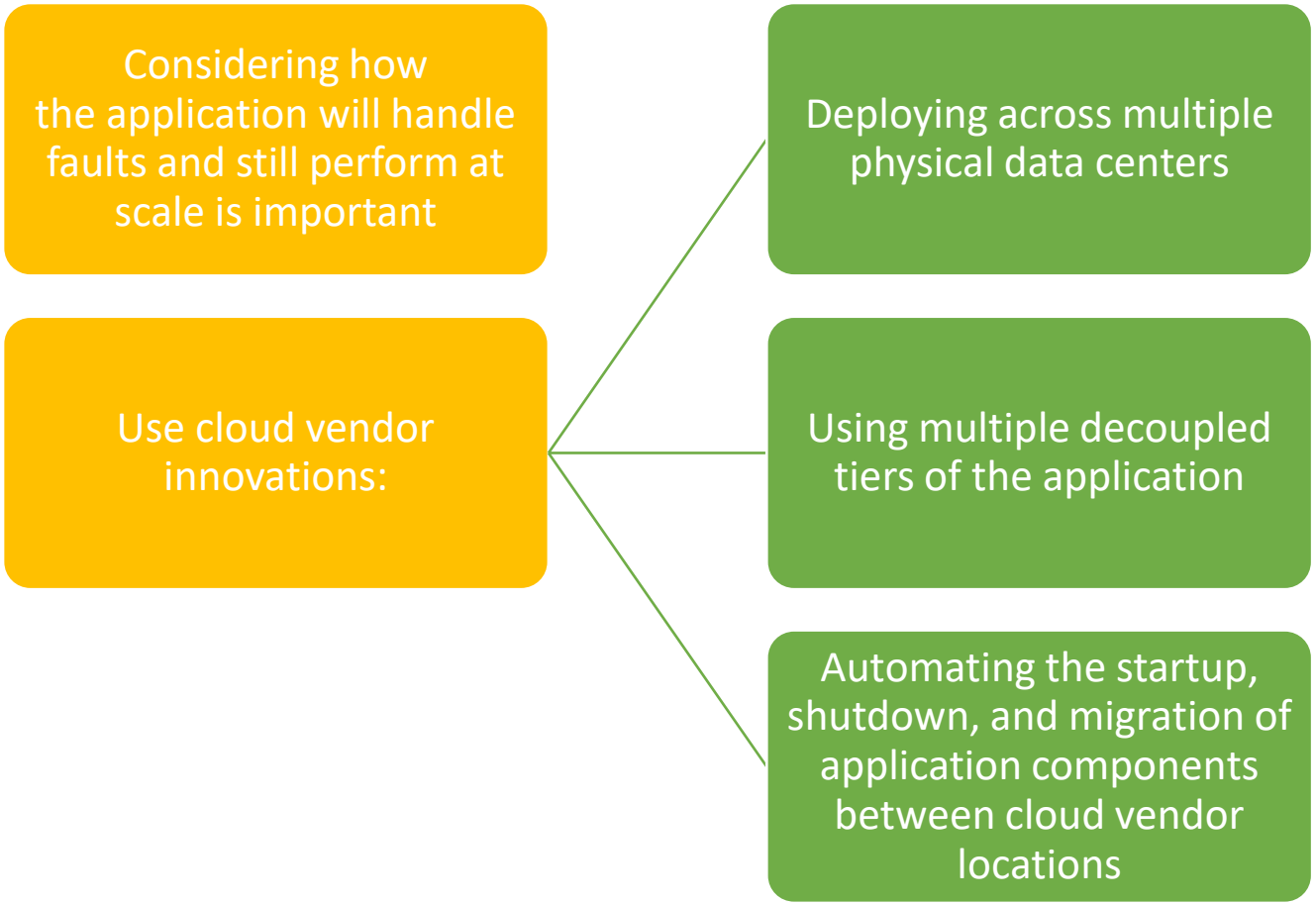
Parallelization

Designing an application that can execute distinct processes in parallel with other parts of the application => have the performance required as it scales up

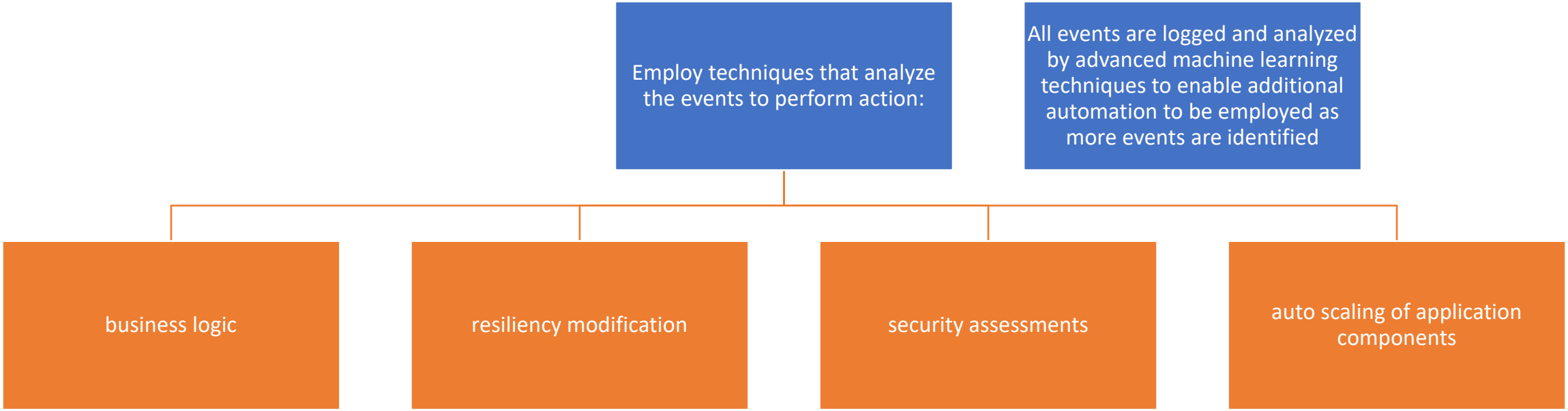
Allowing the same set of functions to execute many times in parallel

Many distinct functions execute in parallel

Resiliency


Considering how the application will handle faults and still perform at scale is important
Use cloud vendor innovations:
Deploying across multiple physical data centers
Using multiple decoupled tiers of the application
Automating the startup, shutdown, and migration of application components between cloud vendor locations

Event-driven



Future-proofed

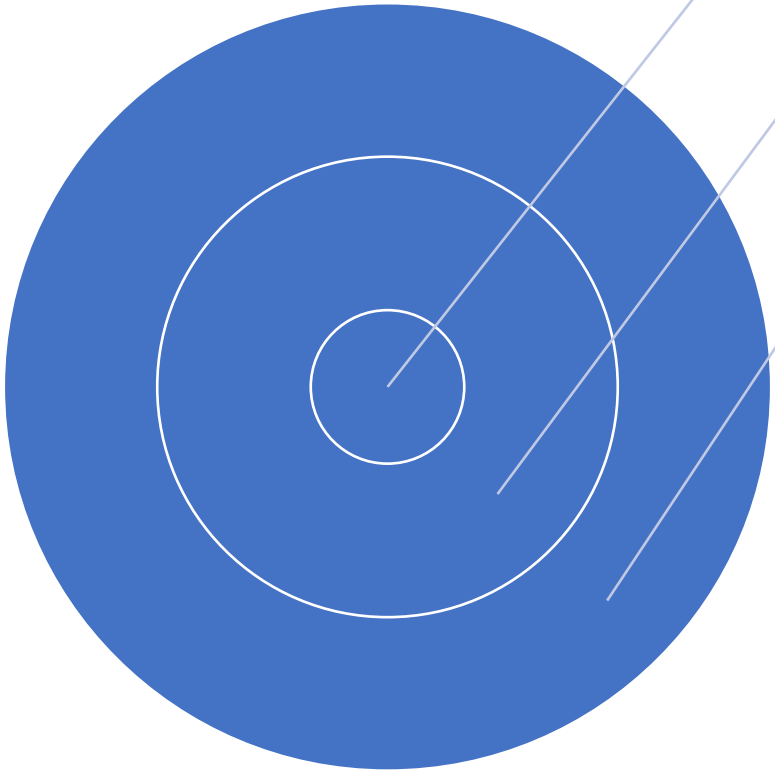
Continue to evolve application as time and innovation moves on

Optimize application through automation and code enhancements

Deliver business results required

Axes 3 – Automation

- Avoid manual operations
- Cloud vendors provide API endpoints
- Adopt infrastructure as code
- Focus on application specific design
- Resource deployment managed by vendor



Management, Configuration, and Deployment

Monitoring, Compliance, and Optimization

Predictive Analytics, AI, and ML

Evolution of Degree of Automation

Environment buildout, resource configuration, and application deployments

Advanced monitoring, scaling, and performance activities

Include auditing, compliance, governance, and optimization of the full solution

Use artificial intelligence and machine and deep learning techniques to self-heal and self-direct the system to change its structure based on the current conditions

Environment management, configuration, and deployment

API-driven environment provisioning, system configuration, and application deployments

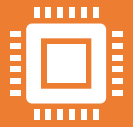
Use code to handle these repeatable tasks

Ensure that agility and consistency are a constant in the process, allowing systems to be deployed often, and following the design requirements exactly

Immutable infrastructure - the system components are never updated, but replaced with the new version or configuration every time (configuration drifts, prove governance, reduce security issues)

CICD -code check-in, automated compiling with code analysis, packaging, and deployment, to specific environments with different hooks or approval stops to ensure a clean deployment.

Monitoring



Cloud native architectures that use complex services and span across multiple geographic locations require the ability to change often based on usage patterns and other factors

Mature cloud vendors have monitoring services natively integrated to their other services that can capture metrics, events, and logs of those services that would otherwise be unobtainable

Using these monitoring services to trigger basic events that are happening is a type of automation that will ensure overall system health (auto-scaling)

Automated compliance of environment and system configuration

Incorporating automation to perform constant compliance audit checks across system components shows a high level of maturity on the automation axis

Compared against previous views of the environment to ensure that configuration drift has not happened

Current snapshot can be compared against desired and compliant configurations that will support audit requirements in regulated industries

Optimization of costs

Before cloud => significant amount of over-provisioned capacity

Automated optimization can be used to constantly check all system components and, using historical trends, understand if those resources are over-or under-utilized

Auto scaling

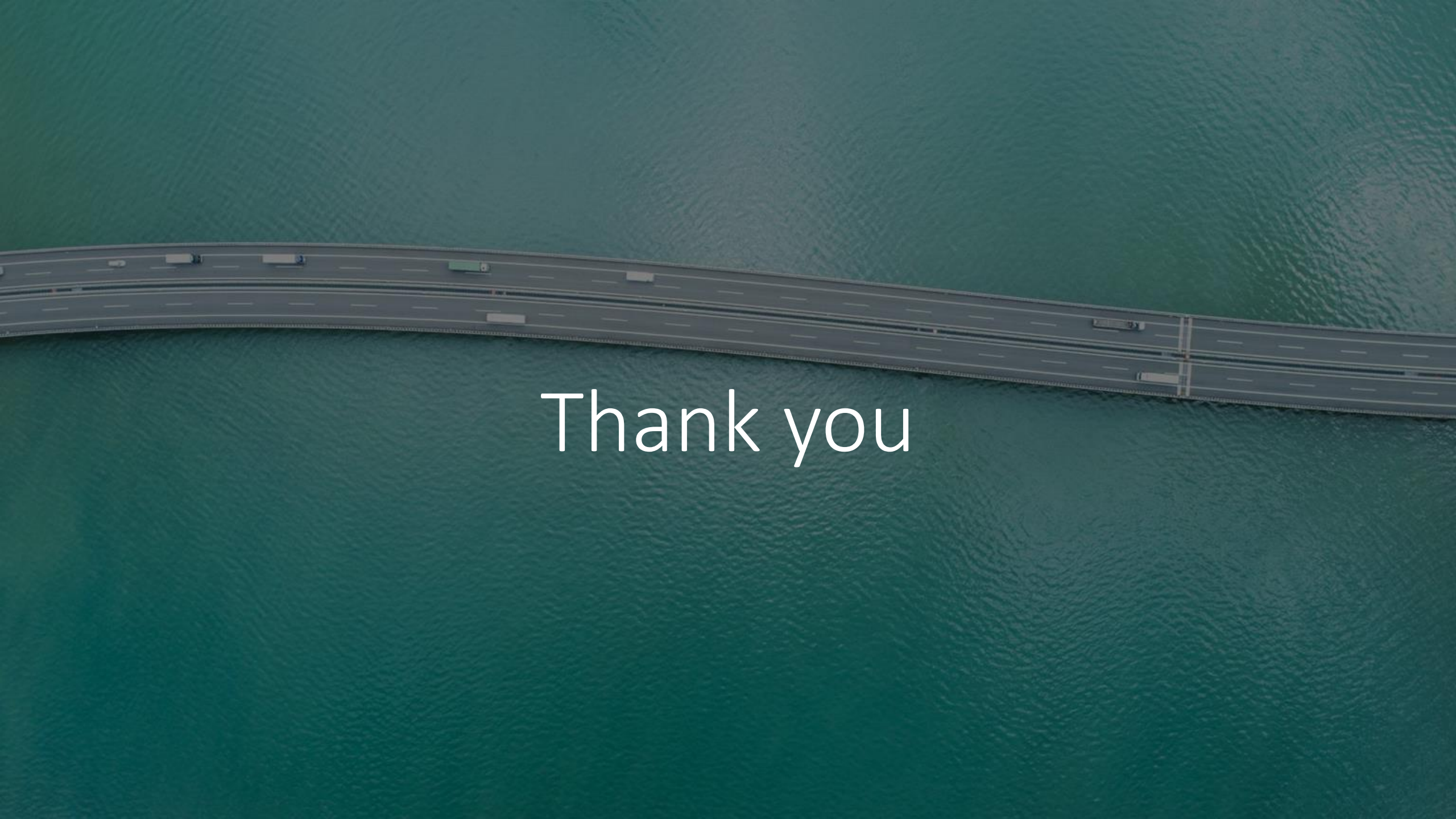
Run an automated process to check running instances and turned unused instances off

Shut-down development environments on nights and weekends

Logs as streams of events that should be analyzed in real time for anomalies of any kind

Predictive analytics, artificial intelligence, machine learning, and beyond

- Analyze generated data and act upon
- Use AI/ML to predict how events could impact the system
- Proactively adjust before security or performance problems, or business degradation
- Show correlation that could not be seen by a human reviewing the same datasets
- Cutting edge of how a mature cloud native architecture can be designed


Thank you